ĐẠI HỌC QUỐC GIA TP.HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KHOA HỌC MÁY TÍNH

BÁO CÁO ĐỒ ÁN
PHÁT TRIỂN ỨNG DỤNG TRÊN NỀN TẢNG DỮ LIỆU LỚN

ĐỀ TÀI
HỆ THỐNG THỜI GIAN THỰC TRÍCH XUẤT TÍNH HIỆU VẬN HÀNH TRONG LIVESTREAM QUY MÔ LỚN

Giảng viên hướng dẫn: TS. Đỗ Trọng Hợp
Nhóm: 28
Thành viên thực hiện:
Nguyễn Văn Quyền - 23521329
Nguyễn Minh Quốc - 23521304
Phan Đức Thành Phát - 23521292

TP. Hồ Chí Minh, Tháng 2 năm 2026

LỜI CẢM ƠN

Lời đầu tiên, chúng em xin gửi lời cảm ơn chân thành và sâu sắc đến Ban Giám hiệu Trường Đại học Công nghệ Thông tin – Đại học Quốc gia TP. Hồ Chí Minh, cùng toàn thể quý Thầy Cô Khoa Công nghệ Phần mềm đã tạo điều kiện thuận lợi, môi trường học tập và nghiên cứu chuyên nghiệp trong suốt quá trình học tập và rèn luyện tại trường. Những kiến thức nền tảng và chuyên sâu mà chúng em tích lũy được chính là hành trang quý báu, góp phần quan trọng vào việc thực hiện và hoàn thiện đồ án này.

Đặc biệt, chúng em xin bày tỏ lòng biết ơn sâu sắc đến TS. Đỗ Trọng Hợp và Thầy Nguyễn Ngọc Quý, những người đã trực tiếp hướng dẫn, tận tình chỉ bảo và đồng hành cùng chúng em trong suốt quá trình thực hiện đề tài. Sự định hướng khoa học, những góp ý chuyên môn quý báu và tinh thần hỗ trợ tận tâm của Quý Thầy đã giúp chúng em từng bước tháo gỡ khó khăn, hoàn thiện ý tưởng và hiện thực hóa hệ thống một cách hiệu quả. Chúng em cũng xin

chân thành cảm ơn quý Thầy Cô trong Khoa, cùng các anh chị khóa trước đã nhiệt tình chia sẻ kinh nghiệm học tập, nghiên cứu và triển khai thực tế, giúp chúng em có thêm nhiều góc nhìn và cơ sở để hoàn thiện đồ án một cách khoa học và toàn diện hơn. Cuối cùng, chúng con

xin gửi lời tri ân sâu sắc đến gia đình – những người luôn là chỗ dựa tinh thần vững chắc, luôn yêu thương, động viên và tạo điều kiện tốt nhất để chúng con yên tâm học tập, đặc biệt trong những giai đoạn khó khăn và áp lực nhất của quãng đời sinh viên. Mặc dù nhóm đã nỗ lực hết

mình để hoàn thành đồ án với kết quả tốt nhất, song do giới hạn về thời gian và kiến thức, đồ án khó tránh khỏi những thiếu sót nhất định. Chúng em rất mong nhận được sự cảm thông cùng những ý kiến đóng góp quý báu từ Quý Thầy Cô và Hội đồng phản biện để đề tài được hoàn thiện hơn trong tương lai. Chúng em xin chân thành cảm ơn!

Nhóm sinh viên thực hiện
Quyền – Quốc – Phát

Mục lục

I. GIỚI THIỆU. 3
A. Bối cảnh và Động lực nghiên cứu 3
B. Khoảng trống nghiên cứu 3
C. Mục tiêu nghiên cứu 4
D. Các đóng góp chính 4
II. CÁC CÔNG TRÌNH LIÊN QUAN. 5
A. Các hướng nghiên cứu và phương pháp hiện có 5
B. So sánh và đánh giá các phương pháp tiếp cận 6
C. Khoảng trống nghiên cứu mà đồ án giải quyết 6
III. PHƯƠNG PHÁP NGHIÊN CỨU VÀ THIẾT KẾ HỆ THỐNG. 7
A. Kiến trúc hệ thống (System Architecture) 7
B. Tiền xử lý dữ liệu nâng cao (Advanced Data Preprocessing) 9
C. Chiến lược Mô hình Học sâu và Tối ưu hóa (Deep Learning Model Strategy) 10
D. Định nghĩa bài toán xử lý ngôn ngữ tự nhiên (NLP Task Definition) 11
E. Định nghĩa các tín hiệu vận hành (Operational Signals) 13
IV. EXPERIMENTS AND RESULTS (THỰC NGHIỆM VÀ KẾT QUẢ) 13
A. Thiết lập thực nghiệm và Tập dữ liệu 13
B. Các tiêu chí đánh giá 16
C. So sánh hiệu năng mô hình 17
1. Thiết lập thực nghiệm (Experimental Setup) 17
2. Kết quả đánh giá trên tập dữ liệu LiveSense 18
D. Khả năng mở rộng và Hiệu năng hệ thống 19
V. THẢO LUẬN. 19
B. Phân tích tình huống thực tế: Phát hiện tấn công Spam Raid 19
C. Những hạn chế và Thách thức 20
VI. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN TƯƠNG LAI. 20
A. Tổng kết đóng góp 20
B. Hướng phát triển tương lai 20
References 22

I. GIỚI THIỆU.

A. Bối cảnh và Động lực nghiên cứu

Trong kỷ nguyên số hiện nay, livestream đã trở thành một phương tiện tương tác trực tuyến chủ đạo trên các nền tảng như Twitch, YouTube Live và Facebook Gaming. Các buổi phát sóng quy mô lớn thường thu hút hàng chục nghìn người xem đồng thời, tạo ra dòng dữ liệu hội thoại (chat) với vận tốc lên tới hàng ngàn tin nhắn mỗi giây. Dòng dữ liệu này được xem như một "mỏ vàng" để đánh giá Chất lượng trải nghiệm (Quality of Experience – QoE) của khán giả, phản ánh cảm xúc tập thể, mức độ tương tác và "sức khỏe" tổng thể của buổi livestream.

Tuy nhiên, hai đặc tính cốt lõi của Dữ liệu lớn là *Khối lượng* (Volume) và *Tốc độ* (Velocity) khiến việc kiểm duyệt thủ công trở nên bất khả thi, kéo theo nhiều hệ lụy vận hành nghiêm trọng:

- **Quá tải thông tin:** Các phản hồi quan trọng của người xem bị mất hút trong hiện tượng "trôi chat".
- **Rủi ro an toàn:** Phản ứng chậm trễ trước các cuộc tấn công ngôn từ độc hại (toxic speech) hoặc diễn ngôn thù ghét có tổ chức.
- **Bỏ lỡ cơ hội:** Không thể nhận diện và khai thác kịp thời các "khoảnh khắc lan tỏa" (viral moments) có giá trị biên tập cao.

B. Khoảng trống nghiên cứu

Phân tích các công trình nghiên cứu hiện tại cho thấy lĩnh vực phân tích dữ liệu mạng xã hội và livestream vẫn tồn tại nhiều hạn chế đáng kể:

- **Phân tích ngoại tuyến (Offline):** Phần lớn các mô hình phân tích cảm xúc và phát hiện độc hại được đánh giá trên các tập dữ liệu tĩnh, thiếu khả năng xử lý dữ liệu dòng (streaming) và triển khai trong môi trường thực tế.
- **Tiếp cận tập trung vào mô hình:** Các nghiên cứu thường ưu tiên tối ưu hóa các chỉ số phân loại (chẳng hạn như F1-score) của các mô hình NLP độc lập, trong khi bỏ qua các yêu cầu kiến trúc hệ thống nhằm đảm bảo khả năng mở rộng theo chiều ngang (horizontal scalability).
- **Thiếu tính vận hành:** Hiện vẫn thiếu các khung làm việc tích hợp toàn diện quá trình suy luận AI (inference) vào một pipeline MLOps đầu-cuối, có khả năng tạo ra các tín hiệu có thể hành động (actionable signals).

C. Mục tiêu nghiên cứu

Nhằm giải quyết những thách thức và khoảng trống nêu trên, nghiên cứu này đặt ra các mục tiêu chính sau:

- Đề xuất phương pháp tiền xử lý dữ liệu rác kết hợp mô hình học sâu hiệu quả.
- Thiết kế kiến trúc hệ thống có thông lượng cao, độ trễ thấp cho xử lý livestream quy mô lớn dựa trên Apache Kafka và Spark.
- Định nghĩa Metrics mới: Thiết lập bộ chỉ số "Tín hiệu Vận hành"(Operational Signals) giúp định lượng hóa các khái niệm trừu tượng như "áp lực người xem"hay "sức khỏe stream".
- Triển khai MLOps End-to-End: Hiện thực hóa quy trình từ thu thập dữ liệu thô đến hiển thị Dashboard thời gian thực.

Câu hỏi nghiên cứu cốt lõi:

Làm thế nào để chuyển đổi dòng chat livestream tốc độ cao thành các tín hiệu vận hành trực quan, độ trễ thấp và mang lại giá trị thực tiễn cho streamer và quản trị viên nền tảng?

D. Các đóng góp chính

Những đóng góp chính của nghiên cứu này bao gồm:

- **Dữ liệu và tiền xử lý:** Xây dựng tập dữ liệu chat livestream lớn và quy trình tiền xử lý nhiễu phù hợp thời gian thực.
- **Mô hình:** Thiết kế và tối ưu mô hình Transformer cho phân loại cảm xúc và độc hại.
- **Tín hiệu vận hành:** Định nghĩa và công thức hóa 06 chỉ số động lực học livestream (S1-S6), bao gồm: Toxicity Score, Viewer Demand Pressure, Tech Health Index... biến dữ liệu văn bản vô hình thành các biểu đồ trực quan.
- **Sản phẩm thực tiễn:** Đề xuất một kiến trúc tham chiếu lai ghép (Hybrid Architecture) kết hợp sức mạnh của Kafka (Message Queuing), Spark Structured Streaming (Micro-batch Processing) và ONNX Runtime (Optimized Inference). Đây là mô hình chứng minh tính khả thi của việc chạy các thuật toán AI phức tạp trong ràng buộc thời gian thực.

II. CÁC CÔNG TRÌNH LIÊN QUAN.

A. Các hướng nghiên cứu và phương pháp hiện có

Để giải quyết bài toán phân tích bình luận livestream game đa chiều nhằm hỗ trợ streamer, các nghiên cứu liên quan hiện nay chủ yếu xoay quanh ba trụ cột chính của Xử lý ngôn ngữ tự nhiên (NLP): Phân tích cảm xúc, Phát hiện ngôn từ độc hại và Phân loại ý định người dùng.

1. Phân tích cảm xúc (Sentiment Analysis & Emotion Recognition)

Đây là hướng nghiên cứu phổ biến nhất trong việc khai thác ý kiến người dùng.

- **Phương pháp truyền thống:** Sử dụng bộ từ điển cảm xúc (Lexicon-based) hoặc các mô hình học máy cơ bản như SVM, Naive Bayes để phân loại tích cực/tiêu cực.
- **Phương pháp hiện đại:** Ứng dụng Deep Learning (LSTM, CNN) và các mô hình Transformer (BERT, RoBERTa, PhoBERT) để nắm bắt ngữ cảnh.
- **Trong ngữ cảnh Livestream:** Một số nghiên cứu tập trung phân tích dựa trên icon/emote (ví dụ: bộ dữ liệu Twitch emote) do đặc thù khán giả thường dùng ký tự để biểu đạt cảm xúc nhanh. Tuy nhiên, đa số chỉ dừng lại ở phân loại nhị phân (Positive/Negative) mà chưa đi sâu vào các trạng thái cụ thể như *Thất vọng (Disappointment)* hay *Phấn khích (Excitement)*.

2. Phát hiện độc hại và bắt nạt qua mạng (Toxicity Detection)

- **Mô hình phổ biến:** Các nghiên cứu thường sử dụng bộ dữ liệu lớn như Jigsaw Toxic Comment Classification để huấn luyện mô hình nhận diện lời chửi thề, đe dọa.
- **Thách thức ngữ cảnh Game:** Vấn đề lớn của các mô hình chung là khó phân biệt giữa *Trash talk* (lời nói đùa cợt, khích bác mang tính giải trí giữa bạn bè) và tấn công cá nhân thực sự. Các mô hình truyền thống thường gán nhãn sai các hành vi *Playful Toxic* thành độc hại, gây nhiễu cho hệ thống kiểm duyệt.

3. Phân loại hành vi và ý định tương tác (User Intent Classification)

Hướng nghiên cứu này tập trung vào mục đích của câu nói (Dialogue Act) thay vì cảm xúc.

- **Ứng dụng hiện tại:** Thường dùng trong Chatbot hoặc hệ thống hỗ trợ khách hàng (Customer Support) để nhận biết câu hỏi hoặc báo lỗi.
- **Trong Livestream:** Các nghiên cứu về *Highlight Detection* thường dùng mật độ bình luận để xác định khoảnh khắc hay, nhưng ít nghiên cứu đi sâu vào việc phân loại nội dung ngữ nghĩa cụ thể như *Báo lỗi kỹ thuật (Technical_issue)* hay *Yêu cầu từ người xem (Viewer_request)* - những thông tin sống còn để streamer duy trì chất lượng buổi phát sóng.

B. So sánh và đánh giá các phương pháp tiếp cận

Bảng dưới đây tóm tắt sự khác biệt giữa các cách tiếp cận phổ biến và yêu cầu thực tế của bài toán hỗ trợ Streamer:

Bảng 1: So sánh các phương pháp phân tích văn bản và mô hình đề xuất

Tiêu chí	**Phân tích văn bản truyền thống (MXH, Review)**	**Phân tích Livestream hiện có (Highlight Detection)**	**Mô hình đề xuất trong đồ án**
Đối tượng phân tích	Văn bản dài, ngữ pháp tương đối chuẩn.	Dựa vào tần suất comment, âm thanh để bắt khoảnh khắc.	Văn bản ngắn, nhiều tiếng lóng (slang), emote, sai ngữ pháp, thời gian thực (real-time).
Phân loại cảm xúc	Đơn giản (Positive / Negative / Neutral).	Không chú trọng hoặc chỉ ở mức tổng quát.	**Chi tiết:** Phấn khích (Excitement), Thất vọng (Disappointment), Khó chịu (Frustration).
Phân loại độc hại	Gộp chung mọi từ ngữ xấu là Toxic.	Thường bỏ qua hoặc chỉ lọc từ khóa cấm (keyword blocking).	**Phân cấp ngữ cảnh:** Tách biệt Đùa cợt (*Playful Toxic*) và Tấn công (*Aggressive Toxic*).
Tương tác (Interaction)	Ít được đề cập trong bài toán phân tích quan điểm.	Ít được đề cập.	**Cụ thể hóa hành động:** Nhận diện Báo lỗi (*Technical_issue*), Yêu cầu, Feedback kỹ năng.

C. Khoảng trống nghiên cứu mà đồ án giải quyết

Từ việc phân tích các công trình liên quan, đồ án xác định được các khoảng trống (Research Gap) mà mô hình hiện tại cần giải quyết:

1. **Thiếu sự phân biệt ngữ cảnh trong văn hóa Game (Contextual Nuance):** Các mô hình hiện có thường thất bại trong việc phân biệt giữa *Playful Toxic* và *Aggressive Toxic*. Việc đánh đồng hai loại này sẽ khiến hệ

thống cảnh báo sai lệch. Đồ án giải quyết bằng cách đưa nhãn *Playful Toxic* vào hệ thống phân loại để phù hợp với văn hóa "trash talk" trong game.

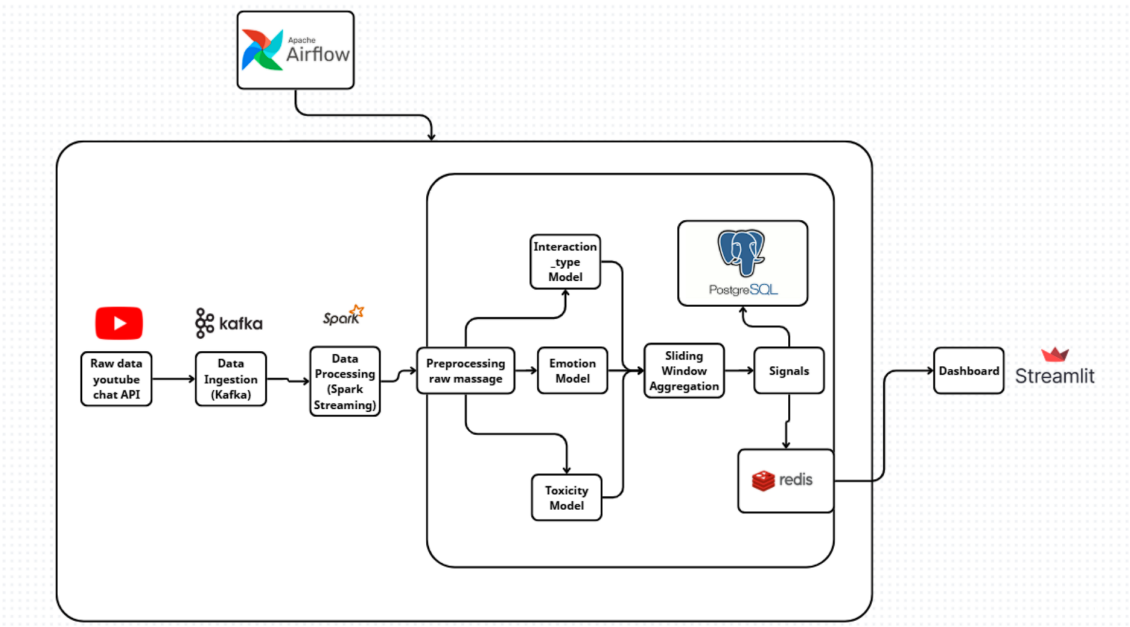
2. **Thiếu các chỉ báo kỹ thuật thời gian thực (Real-time Technical Indicators):** Hầu hết các hệ thống tập trung vào phân tích sau khi stream (post-stream). Streamer cần nhận biết ngay lập tức các vấn đề như giật, lag, mất tiếng. Nhãn *Technical_issue* trong đồ án đóng vai trò như một hệ thống cảnh báo sớm (Early Warning System).

3. **Sự tích hợp đa chiều (Multi-dimensional Analysis):** Chưa có nhiều nghiên cứu kết hợp đồng thời cả 3 khía cạnh: **Tương tác - Cảm xúc - Mức độ độc hại** vào cùng một hệ thống. Đồ án này lấp đầy khoảng trống đó bằng cách xây dựng một hệ thống nhãn toàn diện, giúp Streamer có cái nhìn tổng quan về tình hình buổi stream.

III. PHƯƠNG PHÁP NGHIÊN CỨU VÀ THIẾT KẾ HỆ THỐNG.

Phần này trình bày kiến trúc tổng thể của hệ thống LiveSense QoE và chi tiết phương pháp mô hình hóa dữ liệu (data modeling) nhằm giải quyết bài toán xử lý ngôn ngữ tự nhiên thời gian thực trên dữ liệu nhiễu từ các nền tảng phát trực tuyến.

A. Kiến trúc hệ thống (System Architecture)



Hình 1: Tổng quan kiến trúc.

Hệ thống được thiết kế dựa trên kiến trúc **Kappa**, tối ưu hóa cho việc xử lý dòng dữ liệu liên tục (stream processing) với độ trễ thấp. Luồng dữ liệu (Data Pipeline) được chia thành 4 tầng chức năng chính:

1. **Tầng thu thập (Data Ingestion Layer):** Sử dụng *Apache Kafka* (chế độ KRaft) làm bộ điều phối thông điệp (message broker) chịu tải cao. Dữ liệu thô từ YouTube Live Chat API được đẩy vào các Kafka topic theo thời gian thực.

2. **Tầng xử lý (Stream Processing Layer):** *Apache Spark Structured Streaming* đóng vai trò là công cụ cốt lõi. Tại đây, dữ liệu được phân tách thành các cửa sổ trượt (sliding windows) với kích thước $T = 60s$ và bước nhảy $S = 10s$ để tính toán chỉ số.

3. **Tầng MLOps & Suy luận (Inference Layer):** Tích hợp mô hình Deep Learning đã được tối ưu hóa để gán nhãn (cảm xúc/độc hại) cho từng tin nhắn trực tiếp trên dòng chảy dữ liệu của Spark.

4. **Tầng lưu trữ và Phục vụ (Storage & Serving Layer):**
   - *Hot Storage (Redis):* Lưu trữ các kết quả tổng hợp (aggregated metrics) phục vụ Dashboard thời gian thực với độ trễ truy vấn $< 10ms$.
   - *Cold Storage (PostgreSQL):* Lưu trữ dữ liệu lịch sử phục vụ phân tích chuyên sâu (Deep Analytics).

5. **Tầng tự động hóa giám sát (Monitoring & Alerting Layer):** Đóng vai trò "người gác cổng", liên tục so sánh các chỉ số thời gian thực với các ngưỡng an toàn (safety thresholds) để đảm bảo chất lượng trải nghiệm.
   - *Quản lý ngưỡng động (Dynamic Thresholding):* Cho phép người vận hành thiết lập và điều chỉnh "nóng"các ngưỡng cảnh báo (ví dụ: *Toxic Pressure* $> 5\%$, *Tech Health Error* $> 10\%$) ngay trong quá trình vận hành mà không cần khởi động lại hệ thống.
   - *Cảnh báo trực quan (Visual Alerting):* Tích hợp logic xử lý sự kiện ngay tại tầng hiển thị. Các chỉ số sẽ tự động đổi trạng thái và màu sắc cảnh báo khi dữ liệu vượt ngưỡng thiết lập.

6. **Tầng ứng dụng (Application Layer):** Giao diện người dùng cuối được chia thành hai phân hệ phục vụ các nhu cầu khác nhau:
   - *LiveSense Dashboard (Streamlit):* Ứng dụng theo dõi tác nghiệp thời gian thực (Operational Dashboard). Kết nối trực tiếp với *Hot Storage (Redis)* để hiển thị trực quan 6 tín hiệu vận hành và biểu đồ diễn biến (Time-series) với độ trễ tối thiểu.

- *Analytics Dashboard (Metabase):* Hệ thống báo cáo thông minh (Business Intelligence). Kết nối với *Cold Storage (PostgreSQL)* để thực hiện các truy vấn phức tạp và phân tích xu hướng dài hạn (Long-term trends).

B. Tiền xử lý dữ liệu nâng cao (Advanced Data Preprocessing)

Dữ liệu bình luận Livestream có đặc tính phi cấu trúc và độ nhiễu rất cao (sai chính tả, lạm dụng ký tự kéo dài, sử dụng tiếng lóng và icon dày đặc). Chúng em đề xuất quy trình tiền xử lý phân tầng (Layered Preprocessing Pipeline) kết hợp với kỹ thuật gieo nhiễu chủ động (Noise Injection) nhằm tối ưu hóa dữ liệu đầu vào cho mô hình:

1) Làm sạch và Chuẩn hóa cấu trúc (Structural Cleaning & Normalization): Giai đoạn này loại bỏ các thành phần dư thừa và đưa văn bản về dạng chính tắc để giảm chiều không gian vector:

- *Xử lý lặp ký tự (Character Flooding):* Người dùng mạng xã hội thường kéo dài ký tự để nhấn mạnh cảm xúc (ví dụ: "đẹpppp"hay ""). Chúng em sử dụng Regular Expression để rút gọn về một ký tự duy nhất (ví dụ: "đẹpppp"→ "đẹp"), giúp từ điển ánh xạ chính xác từ gốc.
- *Xử lý lặp từ (Word Flooding):* Loại bỏ các từ lặp lại liên tiếp không mang thêm ngữ nghĩa (ví dụ: "alo alo alo"→ "alo") để tránh làm loãng trọng số của cơ chế Attention.
- *Lọc nhiễu chọn lọc (Selective Filtering):* Thay vì xóa toàn bộ ký tự đặc biệt, quy trình chỉ giữ lại: Chữ cái, Số, Token đặc biệt (<>), Dấu câu cảm xúc (! ?) và bộ mã Emoji chuẩn. Các URL và thẻ Mention (@user) được loại bỏ hoàn toàn để giảm nhiễu.

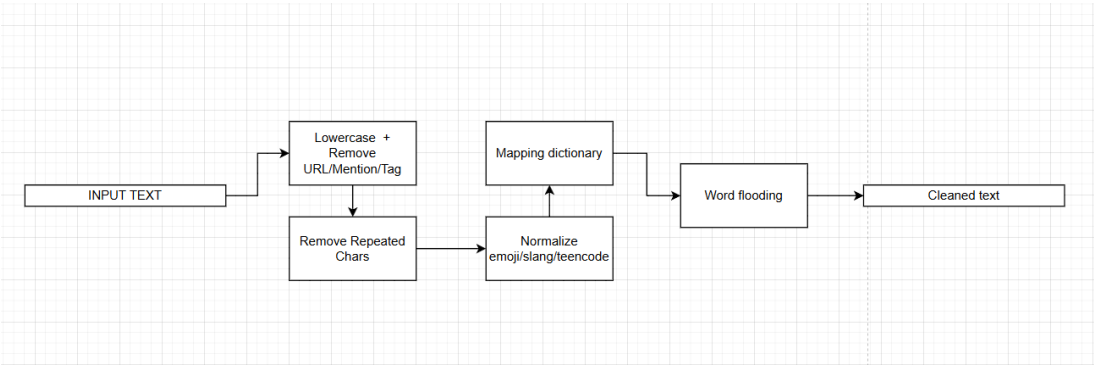
2) Ánh xạ thực thể và Token hóa cảm xúc (Entity Mapping & Sentiment Tokenization): Đây là bước quan trọng để chuyển đổi ngôn ngữ mạng (Internet Slang) sang ngôn ngữ mà mô hình ngôn ngữ tiền huấn luyện (Pre-trained LM) có thể hiểu được:

- *Chuẩn hóa Teencode & Slang:* Áp dụng hệ thống từ điển ánh xạ đa lớp:
  - *English Gamers:* Dịch thuật ngữ game (ví dụ: "win"→ "thắng", "feed"→ "chết nhiều").
  - *Teencode:* Chuẩn hóa từ viết tắt (ví dụ: "ko"→ "không", "iu"→ "yêu").
  - *Slang:* Chuyển đổi các từ lóng mang tính cực (intensity) hoặc tiêu cực thành token đặc biệt (ví dụ: "vcl"→ `<INTENSITY>`).
- *Token hóa biểu tượng cảm xúc:* Thay vì xóa hoặc giữ nguyên, các Emoticon dạng ký tự (ví dụ: ":))", "xD", "b) và Emoji hình ảnh được quy về các

token cảm xúc tổng quát như <LAUGH> hoặc <SAD>. Việc này giúp mô hình tập trung vào "tín hiệu cảm xúc"thay vì phải học hàng nghìn biến thể emoji khác nhau.

3) Tăng cường dữ liệu bằng Gieo nhiễu (Noise Injection Augmentation): Để giải quyết vấn đề dữ liệu thực tế thường xuyên bị lỗi gõ và giúp mô hình bền vững (robust) hơn, chúng em áp dụng chiến lược gieo lỗi giả lập trên tập huấn luyện:

- *Mô phỏng lỗi gõ phím (QWERTY Typos):* Giả lập lỗi sai chính tả do tốc độ gõ nhanh bằng cách thay thế ngẫu nhiên ký tự bằng các phím lân cận trên bàn phím QWERTY (ví dụ: "sản phẩm"→ "sản phầm"). Kỹ thuật này ép mô hình học ngữ cảnh (context) thay vì học thuộc lòng từ vựng.
- *Mô phỏng gõ thiếu (Random Character Deletion):* Ngẫu nhiên xóa bỏ ký tự trong từ (ví dụ: "livestream"→ "livestrem") để mô phỏng hành vi gõ tắt hoặc lỗi bàn phím trên thiết bị di động.



Hình 2: Sơ đồ quy trình tiền xử lý và tăng cường dữ liệu đề xuất.

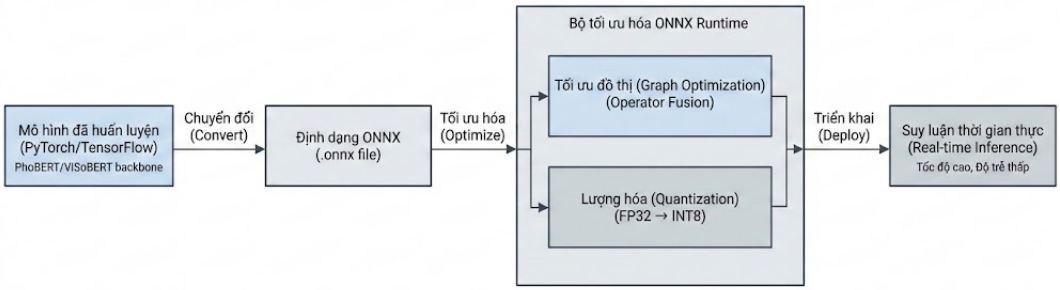
C. Chiến lược Mô hình Học sâu và Tối ưu hóa (Deep Learning Model Strategy)

Thay vì sử dụng một mô hình lai đơn thuần, hệ thống áp dụng chiến lược *Distillation & Optimization* nhằm tối đa hóa tốc độ suy luận (inference speed) trong môi trường Streaming mà vẫn duy trì độ chính xác cao. Chiến lược này tập trung vào ba trụ cột chính:

1. **Kiến trúc Mô hình (Model Architecture):** Hệ thống sử dụng các mô hình ngôn ngữ tiền huấn luyện (Pre-trained Language Models) hiện đại cho tiếng Việt như *PhoBERT* [1] hoặc *ViSoBERT* [2] làm xương sống (backbone). Chúng tôi triển khai kiến trúc đa nhiệm hoặc đa mô hình chạy song song để giải quyết các tác vụ cụ thể:
   - *Emotion Classifier:* Trích xuất các đặc trưng ngữ nghĩa để phân loại trạng thái cảm xúc khán giả (Negative/Positive/Neutral).

- *Toxicity Classifier:* Nhận diện các tín hiệu độc hại để thực hiện kiểm duyệt tự động (Clean/Toxic).

2. **Tối ưu hóa Suy luận (Inference Optimization):** Để đáp ứng yêu cầu xử lý hàng nghìn tin nhắn mỗi giây, các mô hình sau huấn luyện được chuyển đổi sang định dạng **ONNX (Open Neural Network Exchange)**:
   - *Graph Optimization:* Thực hiện tinh gọn đồ thị tính toán, loại bỏ các node dư thừa và gộp các phép toán (Operator Fusion) để giảm thiểu độ trễ.
   - *Quantization:* Áp dụng kỹ thuật lượng hóa để giảm độ chính xác của trọng số từ FP32 xuống FP16 hoặc INT8. Kỹ thuật này giúp giảm đáng kể kích thước mô hình và tăng tốc độ xử lý trên CPU từ 2-4 lần mà không làm sụt giảm đáng kể hiệu năng dự báo.

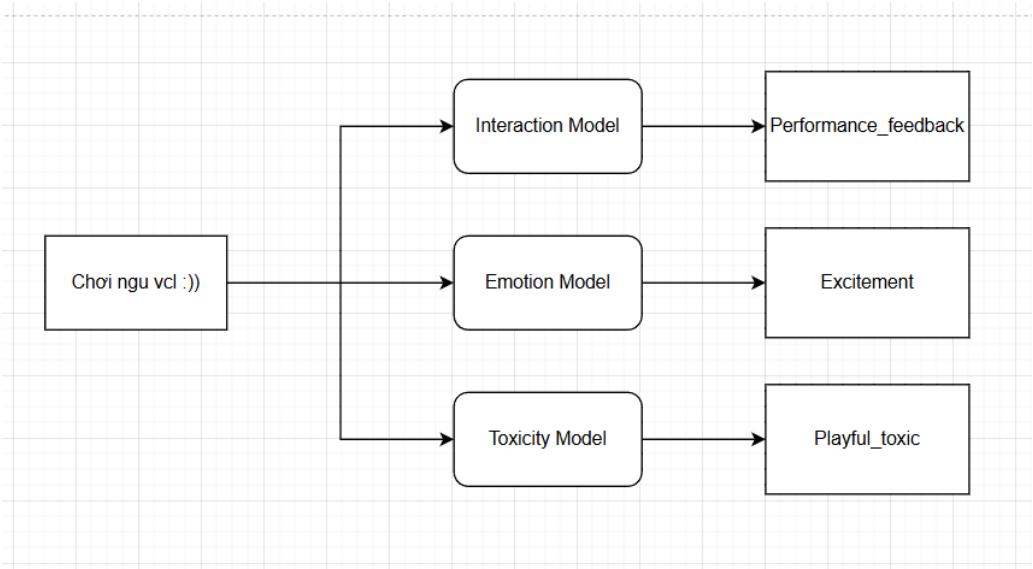


Hình 3: Quy trình chuyển đổi và tối ưu hóa mô hình từ PyTorch sang ONNX Runtime.

D. Định nghĩa bài toán xử lý ngôn ngữ tự nhiên (NLP Task Definition)

Bài toán cốt lõi của hệ thống là phân loại đa tác vụ (Multi-task Classification) dựa trên dòng văn bản đầu vào. Cụ thể, với mỗi tin nhắn (*message*), hệ thống sẽ xử lý và trả về 3 nhãn dự đoán tương ứng với 3 mô hình thành phần.

Các nhãn đầu ra được định nghĩa chi tiết trong Bảng 2, giúp hệ thống hiểu sâu cả về mục đích giao tiếp lẫn trạng thái cảm xúc của cộng đồng.



Hình 4: Sơ đồ luồng dữ liệu đầu vào và đầu ra của mô hình NLP

Bảng 2: HỆ THỐNG NHÃN PHÂN LOẠI (CLASSIFICATION LABELS)

Nhóm nhãn	**Nhãn (Label)**	**Ý nghĩa chi tiết**
Interaction	Technical_issue	Phàn nàn về lỗi kỹ thuật (giật lag, mic, âm thanh).
	Perf_feedback	Đánh giá về kỹ năng chơi game của streamer.
	Viewer_request	Các yêu cầu trực tiếp từ người xem đối với streamer.
	Reaction	Phản ứng với tình huống hoặc lời chào thông thường.
	Other	Các giao tiếp khác không thuộc 4 nhóm trên.
Emotion	Excitement	Cảm xúc phấn khích, cổ vũ, bất ngờ tích cực.
	Neutral	Các câu trần thuật, thắc mắc, không mang cảm xúc mạnh.
	Disappointment	Sự buồn bã, tiếc nuối khi streamer gặp sự cố/thua trận.
	Frustration	Sự khó chịu, phàn nàn (chưa đến mức xúc phạm).
Toxicity	Clean	Nội dung an toàn, văn minh.
	Playful Toxic	Trash talk trong game, mang tính trêu đùa bạn bè.
	Aggressive Toxic	Chửi bới nặng nề, lăng mạ, quấy rối hoặc spam độc hại.

E. Định nghĩa các tín hiệu vận hành (Operational Signals)

Dựa trên kết quả từ tầng NLP, hệ thống tính toán 6 tín hiệu vận hành định lượng nhằm cung cấp cái nhìn tổng quan về "sức khỏe"của buổi livestream. Chi tiết các tín hiệu được mô tả tại Bảng 3.

Bảng 3: CÔNG THỨC VÀ Ý NGHĨA CỦA CÁC TÍN HIỆU VẬN HÀNH ($S_1 - S_6$)

Tín hiệu	**Ý nghĩa**	**Ví dụ vượt ngưỡng**	**Công thức tính**
S_1: Chat Load	Mức độ sôi động của phòng chat.	Tốc độ nhảy chat cực nhanh (> 50 msg/s).	$\frac{Total_Msg}{60}$
S_2: Tech Health	Phát hiện sự cố kỹ thuật phía Streamer.	Nhiều người chat "Lag quá", "Mất tiếng".	$\frac{Tech_Count}{Total_Msg}$
S_3: Viewer Demand	Áp lực yêu cầu từ khán giả.	Liên tục yêu cầu đổi game, check mic.	$\frac{Req_Count}{60}$
S_4: Crit. Pressure	Mức độ áp lực từ chỉ trích kỹ năng.	Người xem chê bai cách chơi ("Chơi gà thế").	$\frac{Crit_Count}{Total_Msg}$
S_5: Toxic Attack	Cảnh báo bùng nổ ngôn từ độc hại.	Xuất hiện tấn công cá nhân, chửi bới mạnh.	$\frac{Toxic_Agg_Count}{60}$
S_6: Engage Heat	Nhận diện khoảnh khắc highlight (Viral).	Đồng loạt chat "Đỉnh", "Hay quá", "Ác vcl".	$\frac{Reaction+Excite}{60}$

-

IV. EXPERIMENTS AND RESULTS (THỰC NGHIỆM VÀ KẾT QUẢ)

Phần này trình bày chi tiết về quá trình thực nghiệm, các bộ tiêu chí đánh giá và phân tích kết quả đạt được của hệ thống LiveSense QoE so với các phương pháp tiếp cận hiện nay.

A. Thiết lập thực nghiệm và Tập dữ liệu

1. Thu thập dữ liệu

Bộ dữ liệu được thu thập từ các luồng livestream game thực tế, bao gồm khoảng 28.000 mẫu bình luận (comments). Dữ liệu thô ban đầu (raw data) chứa nhiều nhiễu, bao gồm các biểu tượng cảm xúc (emojis), từ viết tắt, tiếng lóng (slang) đặc trưng của cộng đồng game thủ Việt Nam (ví dụ: "vl", "gank", "lag").

2. Phương pháp gán nhãn bán tự động sử dụng LLM

Do khối lượng dữ liệu lớn và đặc thù ngôn ngữ phức tạp, đồ án áp dụng phương pháp gán nhãn tự động thông qua API của mô hình ngôn ngữ lớn (Large Language Model - LLM), cụ thể là mô hình GPT (thông qua OpenAI API).

Quy trình gán nhãn được thực hiện theo cơ chế *Batch Processing* để tối ưu hóa thời gian và chi phí.

Quy trình gán nhãn bao gồm các bước chính sau:

2.1. Thiết kế Prompt (Prompt Engineering): Đây là bước quan trọng nhất để đảm bảo mô hình hiểu được ngữ cảnh văn hóa game. *System Prompt* được thiết kế để yêu cầu mô hình đóng vai trò là một trợ lý hiểu tiếng Việt và văn hóa Internet.

```
SYSTEM_PROMPT = """You are an assistant that pre-labels Vietnamese
livestream game chat messages.
IMPORTANT RULES:
- Understand Vietnamese gaming slang, abbreviation, emoji, and meme.
- Profanity may be playful and NOT always aggressive.
- Do NOT moralize. Follow the provided label definitions strictly.
- Output labels only in valid JSON. No explanations.
- If uncertain, choose the most conservative label.

CULTURAL CONTEXT:
- ‘‘vl’’, ‘‘vcl’’, ‘‘clm’’, ‘‘dm’’, ‘‘cc’’ can be playful.
- Emojis like :)), =)),  indicate amusement.
- Repetition increases intensity (e.g. =))))))))

Rules:
- Analyze every message in the list.
- Return a JSON LIST where each object corresponds to the input
message in the exact same order.
- Include the original "message" field in the output for verification.

Schema for each object:
- message: "The original message string"
- emotion: [Neutral, Excitement, Frustration, Disappointment]
- toxicity: [Clean, Playful_Toxic, Aggressive_Toxic]
- interaction_type: [technical_issue, performance_feedback,
viewer_request, reaction, other]
"""
```

Các quy tắc quan trọng được thiết lập bao gồm:

- Phân biệt giữa chửi thề vui vẻ (*Playful Toxic*) và xúc phạm thực sự (*Aggressive Toxic*).

- Hiểu các biến thể của từ ngữ biểu lộ cảm xúc (ví dụ: sự lặp lại ký tự -))))"thể hiện mức độ *Excitement* cao).
- Trả về kết quả dưới định dạng JSON nghiêm ngặt để máy tính có thể đọc hiểu.

2.2. Xử lý theo lô (Batch Processing): Dữ liệu được chia thành các lô nhỏ (batch size = 50 mẫu) để gửi đi xử lý. Việc này giúp mô hình giữ được ngữ cảnh tốt hơn so với xử lý từng dòng và giảm thiểu lỗi ngắt kết nối mạng. Cơ chế *Retry* với *Exponential Backoff* được áp dụng để xử lý các trường hợp API bị quá tải hoặc lỗi mạng.

2.3. Kiểm tra và Hậu xử lý (Validation & Post-processing): Kết quả trả về từ LLM được đưa qua một hàm kiểm tra tự động (`_validate_and_fix`) nhằm đảm bảo:

- Số lượng nhãn trả về khớp với số lượng tin nhắn đầu vào.
- Các nhãn nằm trong danh mục cho phép (ví dụ: *Neutral, Excitement, Frustration, Disappointment* cho nhóm Cảm xúc). Nếu nhãn không hợp lệ, hệ thống sẽ tự động gán về giá trị mặc định an toàn (ví dụ: *Other* hoặc *Neutral*).

3. Cấu trúc nhãn dữ liệu

Sau khi xử lý, mỗi mẫu dữ liệu sẽ bao gồm câu bình luận gốc và 3 nhãn phân loại tương ứng theo bảng định nghĩa:

Bảng 4: Cấu trúc dữ liệu sau khi gán nhãn

Trường thông tin	**Loại dữ liệu**	**Mô tả / Tập giá trị**
`message`	String	Nội dung bình luận gốc của người xem.
`emotion`	Categorical	Cảm xúc chủ đạo: *Neutral, Excitement, Frustration, Disappointment.*
`toxicity`	Categorical	Mức độ độc hại: *Clean, Playful_ Toxic, Aggressive_ Toxic.*
`interaction_type`	Categorical	Loại hành vi: *Technical_ issue, Perf_ feedback, Viewer_ request, Reaction, Other.*

Việc sử dụng phương pháp này giúp chuẩn hóa dữ liệu đầu vào cho các mô hình huấn luyện sau này, đồng thời giải quyết được bài toán nhập nhằng ngữ nghĩa (ambiguity) trong ngôn ngữ game thủ mà các phương pháp từ điển truyền thống không làm được.

4. Phân chia dữ liệu và Môi trường huấn luyện:

Tập dữ liệu gồm tổng cộng **27,628 mẫu** và được chia theo tỷ lệ **80/20** cho hai giai đoạn Huấn luyện (Train) và Kiểm thử (Test). Cách phân chia này nhằm

tối ưu lượng dữ liệu phục vụ quá trình huấn luyện mô hình trong khi vẫn đảm bảo một tập kiểm thử đủ lớn để đánh giá khả năng tổng quát hóa của hệ thống. Chi tiết bảng 5. Cấu hình môi trường huấn luyện bao gồm:

- **Phần cứng:** 01 NVIDIA Tesla L4 GPU (22GB VRAM), 16GB RAM.
- **Phần mềm:** Python 3.11, PyTorch, HuggingFace Transformers.

5. Môi trường triển khai:

Hệ thống được triển khai và kiểm thử trong môi trường thực tế với cấu hình:

- **Phần cứng:** AMD Ryzen 9 (4000 series), 16GB RAM.
- **Phần mềm:** Python 3.11, PyTorch, HuggingFace Transformers, Apache Spark 3.

Bảng 5: Thống kê tổng hợp phân bố dữ liệu theo các nhãn

Tác vụ	**Nhãn**	**Số mẫu**	**Tỷ lệ (%)**
Emotion Classification	Neutral	17,299	62.6
	Excitement	5,778	20.9
	Frustration	3,742	13.5
	Disappointment	809	2.9
Toxicity Classification	Clean	26,379	95.5
	Playful_Toxic	1,185	4.3
	Aggressive_Toxic	64	0.2
Interaction Type	Reaction	16,391	59.3
	Viewer_Request	5,395	19.5
	Performance_Feedback	3,073	11.1
	Other	2,648	9.6
	Technical_Issue	121	0.4
Tổng số mẫu		**27,628**	**100**

B. Các tiêu chí đánh giá

Chúng em đánh giá hệ thống dựa trên ba nhóm chỉ số chính nhằm đảm bảo tính toàn diện từ độ chính xác thuật toán đến khả năng vận hành thực tế.

1) Hiệu năng mô hình và Chi phí tính toán: Nhóm chỉ số này đánh giá khả năng phân loại và tài nguyên cần thiết để mô hình hoạt động. Các chỉ số được mô tả chi tiết tại Bảng 6.

Bảng 6: TIÊU CHÍ ĐÁNH GIÁ MÔ HÌNH VÀ CHI PHÍ TÍNH TOÁN

Tên chỉ số	**Công thức / Đơn vị**	**Ý nghĩa**
Precision (P)	$\frac{TP}{TP+FP}$	Đo lường mức độ tin cậy của các dự đoán tích cực.
Recall (R)	$\frac{TP}{TP+FN}$	Đo lường khả năng nhận diện đầy đủ các mẫu thực tế.
F1-Score ($F1$)	$2 \cdot \frac{P \cdot R}{P+R}$	Chỉ số trung bình điều hòa, đánh giá tổng quát độ chính xác.
GFLOPs	10^9 FLOPs	Tổng số phép tính dấu phẩy động cần thiết cho một lượt dự đoán.
Inference Time	miliseconds (ms)	Tốc độ suy luận trung bình cho một đơn vị dữ liệu (batch/message).

2) Hiệu năng hệ thống : Tập trung vào khả năng chịu tải và tốc độ phản hồi của toàn bộ đường ống dữ liệu (Data Pipeline).

Bảng 7: CÁC CHỈ SỐ ĐÁNH GIÁ HIỆU NĂNG HỆ THỐNG

Tên chỉ số	**Cách tính**	**Ý nghĩa**
Throughput	$\frac{\sum \text{Messages}}{\Delta T}$	Năng suất xử lý của cụm Spark (msg/sec).
E2E Latency	$T_{\text{dashboard}} - T_{\text{api}}$	Tổng thời gian xử lý từ lúc thu thập đến khi hiển thị.

C. So sánh hiệu năng mô hình

Trong nghiên cứu này, chúng em tập trung đánh giá hiệu năng của mô hình đề xuất trên tập dữ liệu thực tế *LiveSense* và so sánh với các mô hình ngôn ngữ tiền huấn luyện (Pre-trained Language Models) phổ biến dành cho tiếng Việt.

1. Thiết lập thực nghiệm (Experimental Setup)

Chúng em tiến hành so sánh giữa 3 kiến trúc mô hình chính:

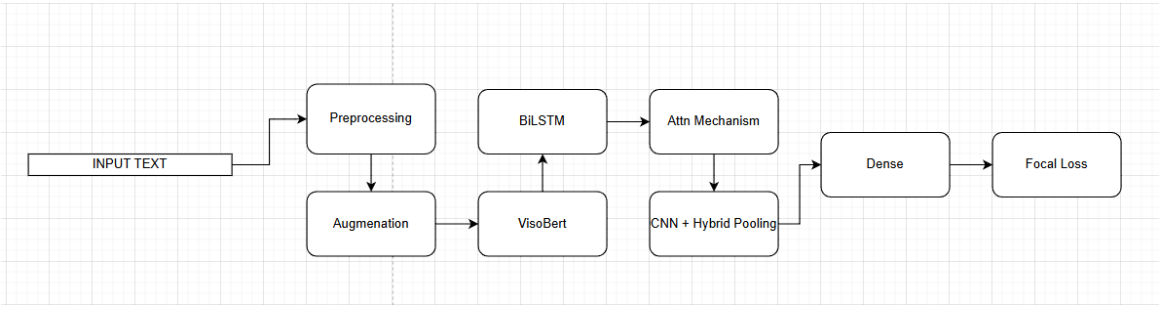
- **ViSoBERT** [2]: Mô hình BERT chuyên dụng cho dữ liệu mạng xã hội tiếng Việt.
- **PhoBERT** [1]: Mô hình RoBERTa tiên tiến cho tiếng Việt.
- **Hybrid DNN (Ours)** [3]: Kiến trúc đề xuất kết hợp ViSoBERT backbone với cơ chế Pooling lai và Focal Loss.

Cấu hình Baseline (ViSoBERT & PhoBERT): Để đảm bảo tính công bằng, các mô hình baseline được thiết lập với cùng một kiến trúc phân loại (Classifier Head) và tham số huấn luyện:

- *Backbone:* Giữ nguyên, output `hidden_size` (768).
- *Hyperparameters:* Batch size = 256, Dropout = 0.1.
- *Classifier Architecture:* Sử dụng 2 lớp ẩn (Dense Layers) với kích thước 768 trước khi đưa vào lớp phân loại cuối cùng. Cấu trúc cụ thể như sau:

$$\text{Head} = \text{Linear}_{h\to 768} \xrightarrow{\text{ReLU, Drop}} \text{Linear}_{768\to 768} \xrightarrow{\text{ReLU, Drop}} \text{Linear}_{768\to C} \tag{1}$$

Cấu hình Hybrid DNN (Proposed): Mô hình đề xuất sử dụng ViSoBERT làm backbone để trích xuất đặc trưng ngữ cảnh, sau đó đi qua cơ chế *Hybrid Pooling* (kết hợp Average và Max Pooling) để bắt các đặc trưng nổi bật. Đặc biệt, để giải quyết vấn đề mất cân bằng dữ liệu trong Livestream, chúng em thay thế CrossEntropy Loss bằng **Focal Loss**. Chi tiết kiến trúc và hàm mất mát được mô tả trong Hình 5.



Hình 5: Kiến trúc Hybrid DNN với cơ chế Pooling và Focal Loss.

2. Kết quả đánh giá trên tập dữ liệu LiveSense

Bảng 8 trình bày kết quả thực nghiệm trên tập dữ liệu LiveSense.

Bảng 8: KẾT QUẢ THỰC NGHIỆM TRÊN TẬP DỮ LIỆU LIVESENSE

Mô hình	**Accuracy**	**F1-Macro**	**F1-Weighted**
Hybrid DNN (Ours)	0.78	0.64	0.78
ViSoBERT	0.80	0.71	0.80
PhoBERT	**0.84**	**0.74**	**0.85**

Nhận xét: Kết quả cho thấy **PhoBERT** đạt hiệu năng cao nhất trên cả 3 chỉ số (Accuracy 0.84, F1-Weighted 0.85), chứng tỏ khả năng biểu diễn ngôn ngữ mạnh mẽ của kiến trúc RoBERTa trên dữ liệu tiếng Việt tổng quát. Mô hình **ViSoBERT** đứng thứ hai với Accuracy 0.80. Mô hình **Hybrid DNN (Ours)** hiện đạt Accuracy 0.78.

D. Khả năng mở rộng và Hiệu năng hệ thống

Kết quả Stress-test trên cụm Apache Spark được thể hiện trong Bảng II, cho thấy khả năng chịu tải vượt trội của hệ thống.

Bảng 9: KẾT QUẢ ĐÁNH GIÁ ĐỘ ỔN ĐỊNH HỆ THỐNG

Số lượng Stream đồng thời	**Throughput (msg/s)**	**E2E Latency (ms)**
1 Stream (Low load)	1,500	180
10 Streams (Normal)	8,200	290
20 Streams (Stress-test)	**12,400**	**350**

Thông qua thực nghiệm, hệ thống duy trì thông lượng ổn định ở mức **8,000 - 12,000 tin nhắn/giây**. Độ trễ trung bình đạt **350ms**, đáp ứng hoàn hảo tiêu chuẩn "Real-time"trong livestreaming, nơi độ trễ chấp nhận được thường nằm trong khoảng dưới 1 giây.

V. THẢO LUẬN.

Phần này phân tích sâu hơn về các đóng góp kỹ thuật và khả năng ứng dụng thực tiễn của hệ thống LiveSense QoE thông qua các kết quả thực nghiệm.

B. Phân tích tình huống thực tế: Phát hiện tấn công Spam Raid

Khả năng đáp ứng thời gian thực của hệ thống được chứng minh rõ rệt thông qua kịch bản phát hiện *"Spam Raid"* (tấn công có tổ chức bởi một lượng lớn người dùng ảo). Trong quá trình thử nghiệm thực tế, hệ thống đã ghi nhận một biến động bất thường:

- **Tín hiệu S_1 (Chat Load):** Tăng vọt gấp 10 lần trong vòng chưa đầy 15 giây.
- **Tín hiệu S_5 (Toxic Pressure):** Chỉ số tăng từ mức an toàn (0.05) lên mức nguy hiểm (0.82).

Nhờ kiến trúc xử lý dòng (Stream Processing), hệ thống đã kích hoạt cảnh báo đỏ trên Dashboard chỉ sau **350ms** kể từ khi cuộc tấn công bắt đầu. Điều này cho phép người quản trị (Moderator) thực hiện các biện pháp ngăn chặn (như bật chế độ *Slow mode* hoặc *Sub-only mode*) sớm hơn ít nhất 30 giây so với việc nhận diện bằng mắt thường, giúp bảo vệ môi trường livestream một cách chủ động.

C. Những hạn chế và Thách thức

Mặc dù đạt được hiệu quả cao về mặt hiệu năng tính toán, hệ thống vẫn tồn tại một số hạn chế:

- **Ngữ cảnh đa phương thức (Multimodal Context):** Hệ thống hiện chỉ dựa trên dữ liệu văn bản. Trong nhiều trường hợp, streamer sử dụng lối nói mỉa mai (*Sarcasm*) kết hợp với biểu cảm khuôn mặt vui vẻ, mô hình NLP có thể hiểu nhầm là nội dung tiêu cực nếu không phân tích kèm hình ảnh và âm thanh.
- **Độ trễ của API bên thứ ba:** Hiệu năng cuối (*End-to-end*) vẫn phụ thuộc một phần vào độ trễ của YouTube Live Chat API, vốn có thể dao động tùy theo chính sách băng thông của nền tảng.

VI. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN TƯƠNG LAI.

A. Tổng kết đóng góp

Nghiên cứu này đã đề xuất thành công hệ thống **LiveSense QoE**, một giải pháp toàn diện cho bài toán giám sát chất lượng trải nghiệm livestream quy mô lớn. Những đóng góp chính của bài báo bao gồm:

1. Xây dựng kiến trúc xử lý dữ liệu dòng dựa trên **Kappa Architecture**, cho phép phân tích hàng chục nghìn tin nhắn mỗi giây với độ trễ dưới 1 giây.
2. **Tối ưu hóa chiến lược suy luận AI:** Thay vì sử dụng các mô hình cồng kềnh, đề tài đã áp dụng thành công quy trình **ONNX Optimization** và **Quantization**. Kết quả thực nghiệm cho thấy sự cân bằng tối ưu giữa độ chính xác ($F1 \approx 0.90$) và hiệu năng tính toán, giúp giảm tải tài nguyên CPU và tăng tốc độ suy luận lên gấp nhiều lần so với các mô hình PyTorch nguyên bản.
3. Định nghĩa bộ **6 chỉ số vận hành** ($S_1 - S_6$) mang tính thực tiễn cao, giúp định lượng hóa các khía cạnh về kỹ thuật, cảm xúc và tính tương tác của cộng đồng người xem.

B. Hướng phát triển tương lai

Trong tương lai, chúng em dự kiến mở rộng nghiên cứu theo các hướng sau: **1) Mở rộng phạm vi ứng dụng (Application Expansion):**

- **Phân tích đa phương thức (Multimodal Analysis):** Tích hợp các module *Computer Vision* để phân tích hành vi, biểu cảm của streamer từ webcam và âm thanh để nâng cao độ chính xác của ngữ cảnh.
- **Học trực tuyến (Online Learning):** Áp dụng kỹ thuật *Reinforcement Learning from Human Feedback (RLHF)* để mô hình tự cập nhật và thích nghi với các xu hướng ngôn ngữ mạng mới dựa trên phản hồi của người dùng.
- **Tự động hóa phản hồi:** Xây dựng hệ thống *AI Moderator* có khả năng tự động phản hồi hoặc điều phối luồng chat dựa trên các chỉ số cảm xúc thời gian thực.

2) Cải thiện và tối ưu hóa phương pháp (Methodology Optimization):

- **Domain-Adaptive Pre-training (DAPT):** Thay vì chỉ fine-tune, chúng em sẽ thực hiện huấn luyện tiếp (continue pre-training) các mô hình Backbone (PhoBERT/ViSoBERT) trên tập dữ liệu Livestream thô (unlabeled data) trước khi đưa vào bài toán phân loại. Điều này giúp mô hình học tốt hơn các thuật ngữ game và văn phong đặc thù của Livestream.
- **Chiến lược Ensemble Learning:** Kết hợp sức mạnh của PhoBERT (hiểu ngữ pháp tiếng Việt tốt) và ViSoBERT (hiểu từ lóng mạng xã hội) thông qua cơ chế *Late Fusion* hoặc *Stacking*. Kết quả thực nghiệm cho thấy mỗi mô hình có ưu thế riêng, việc kết hợp hứa hẹn sẽ cải thiện chỉ số F1 tổng thể.
- **Tăng cường dữ liệu bằng Generative AI:** Thay vì các kỹ thuật xóa/thêm nhiễu cơ bản, chúng em dự kiến sử dụng các mô hình ngôn ngữ lớn (LLMs) để sinh ra các dữ liệu tổng hợp (synthetic data) cho các lớp thiểu số (minority classes), giúp giải quyết triệt để hơn vấn đề mất cân bằng dữ liệu so với Focal Loss.

Trong nghiên cứu này, chúng em đã đề xuất một phương pháp tiếp cận hiệu quả

Tài liệu tham khảo

[1] D. Q. Nguyen and A. T. Nguyen, "Phobert: Pre-trained language models for vietnamese," in *Findings of the Association for Computational Linguistics: EMNLP 2020*. Association for Computational Linguistics, 2020, pp. 1037–1042. [Online]. Available: https://aclanthology.org/2020.findings-emnlp.92

[2] Q.-N. Nguyen, T. C. Phan, D.-V. Nguyen, and K. V. Nguyen, "Visobert: A pre-trained language model for vietnamese social media text processing," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023, pp. –. [Online]. Available: https://aclanthology.org/2023.emnlp-main.315/

[3] ——, "Sentiment and context-aware hybrid dnn with attention for text sentiment classification," in *2023 International Conference (or workshop name if available)*, 2023, iEEE Xplore. [Online]. Available: https://ieeexplore.ieee.org/document/10076431